

Module 5 Week B — Stretch: From Notebook to Production Script

Honors Track. This is an Honors Track stretch assignment. Stretch assignments are for learners who have completed all core assignments, are On Track or Advanced, and are attending consistently. If you are behind on core work, focus there first.

Prerequisite: Integration 5B base + at least Integration 5B Challenge Tier 1 (Threshold Optimization for Deployment). This stretch refactors your pipeline into a production CLI tool — you need the threshold optimization experience from Tier 1 to implement the `--dry-run` validation and understand what a production-ready pipeline requires beyond “it runs.”

The Challenge

Take the complete model comparison pipeline from Integration 5B and refactor it into a production-quality command-line script. The goal: someone who has never seen your code should be able to run it from the command line and produce the full comparison report without reading the source.

This is a professional engineering skill that separates notebook prototyping from production delivery. In industry, models are rarely run from Jupyter notebooks. They are wrapped in CLI (command-line interface) tools, scheduled as pipeline stages, or deployed as services. The ability to refactor exploratory code into a clean, configurable, well-documented script is what makes a data scientist’s work usable by the rest of the engineering team.

What You’ll Learn

- Refactoring notebook code into a modular Python script
- Building a CLI with `argparse` (arguments, flags, help text, defaults)
- Replacing `print` statements with structured logging (`logging` module)
- Designing a `--dry-run` mode for configuration validation
- Writing user-facing documentation (README with usage examples)

Outcome

Your deliverable is a standalone repository containing:

Artifact	Description
<code>compare_models.py</code> (or similar)	Production CLI script that runs the full model comparison pipeline
<code>README.md</code>	Documents installation, usage, arguments, and example commands
Sample output	At least one sample run showing the script’s output (saved to the output directory)

CLI arguments the script must accept:

Argument	Type	Description
<code>--data-path</code>	Required	Path to the input dataset (CSV)
<code>--output-dir</code>	Optional (default: <code>./output</code>)	Directory where results and plots are saved
<code>--n-folds</code>	Optional (default: 5)	Number of cross-validation folds
<code>--random-seed</code>	Optional (default: 42)	Random seed for reproducibility
<code>--dry-run</code>	Flag	Validates the data and prints the pipeline configuration without training any models

Constraints:

- Use `argparse` for CLI argument parsing (not `click` , `typer` , or `sys.argv` parsing)
- Replace all `print` statements with Python’s `logging` module – use appropriate log levels (`INFO` for progress, `WARNING` for potential issues, `DEBUG` for detailed diagnostics)
- The `--dry-run` flag should load and validate the data (check for expected columns, report shape and class distribution) and print the full pipeline configuration (models to compare, CV settings, output paths) without fitting any models
- The script should create the output directory if it does not exist
- All results (metrics table, plots, comparison summary) should be saved to `--output-dir` , not displayed interactively
- The script should exit with a non-zero exit code if the data file is not found or validation fails
- The `README.md` should include: a one-sentence description, installation/dependency instructions, usage with all arguments explained, and at least two example commands (a normal run and a dry run)

Design guidance:

- Structure the script with clear functions: `load_data()` , `validate_data()` , `train_and_evaluate()` , `save_results()` , `main()` . The exact function decomposition is yours to design.
- The script should be importable as a module (use `if __name__ == '__main__':`) so functions can be tested independently.
- Think about what a user needs to know if something goes wrong. Logging messages should be informative: “Loading data from /path/to/file.csv (12,345 rows, 15 columns)” is useful. “Step 1 complete” is not.

Tips

- Start by listing every cell in your Integration 5B notebook that contributes to the final pipeline. Not all notebook cells belong in a production script – exploratory analysis, scratch work, and visualization experiments can be dropped or moved to a separate analysis script.
- `argparse` generates `--help` text automatically from your argument definitions. Write clear `help` strings for each argument – this is your script’s user interface.
- Test your script by running it from the command line, not from a notebook. Common issues: hardcoded file paths, missing imports, reliance on notebook state.
- A good `--dry-run` mode builds confidence that the script will work before committing to a long training run. In production, this pattern saves time and compute.

Submission

- Create a standalone repository for this stretch assignment (not a GitHub Classroom repo – create your own)
- Push your script, README, and sample output to the repository
- Paste your repository URL into TalentLMS → Module 5 Week B → Stretch 5B-S2

